Practical no :- 1

Title:-Predict the price of the Uber ride from a given pickup point to the agreed drop-off location. Performfollowing tasks: 1. Pre-process the dataset. 2. Identify outliers. 3. Check the correlation. 4. Implement linear regression and random forest regression models. 5.Evaluatethemodelsandcomparetheirrespectivescoreslike R2,RMSE,etc.

Dataset :- https://www.kaggle.com/datasets/yasserh/uber-fares-dataset

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns


df=pd.read_csv("uber.csv")


df , df.head() , df.tail() , df.describe() , df.info() ,df.shape

df.isnull().sum()


df["pickup_datetime"] = pd.to_datetime(df["pickup_datetime"])


df.dropna(inplace=True)


plt.boxplot(df['fare_amount'])


q_low= df["fare_amount"].quantile(0.01)

q_hi= df["fare_amount"].quantile(0.99)

df= df[(df["fare_amount"]< q_hi) & (df["fare_amount"]> q_low)]


df.isnull().sum()


from sklearn.model_selection import train_test_split


X= df.drop("fare_amount", axis=1)
```

```
y= df['fare_amount']

X['pickup_datetime']= pd.to_numeric(pd.to_datetime(X['pickup_datetime']))
X= X.loc[:, X.columns.str.contains('Unnamed')]

x_train, x_test, y_train, y_test= train_test_split(
X, y, test_size=0.2, random_state=1)

from sklearn.linear_model import LinearRegression

lrmodel= LinearRegression()
lrmodel.fit(x_train, y_train)

predict= lrmodel.predict(x_test)

from sklearn.metrics import mean_squared_error
lrmodel.rmse= np.sqrt(mean_squared_error(predict, y_test))
print("RMSE error for the model is", lrmodel.rmse)

from sklearn.ensemble import RandomForestRegressor
rfrmodel = RandomForestRegressor(n_estimators=100, random_state=101)

rfrmodel.fit(x_train, y_train)
rfrmodel_pred= rfrmodel.predict(x_test)

rfrmodel_rmse= np.sqrt(mean_squared_error(rfrmodel_pred, y_test))
print("RMSE value for RandomForestis:", rfrmodel_rmse)
```

practical no :- 2

Title:-Classify the email using the binary classification method. Email Spam detection has two states: a)Normal State – Not Spam, b) Abnormal State – Spam. Use K-Nearest Neighbors and Support VectorMachine for classification. Analyze their performance.

Dataset link: The emails.csv dataset on the Kaggle
https://www.kaggle.com/datasets/balaka18/email-spam-classification-dataset-csv

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import (

    accuracy_score,

    classification_report,

    confusion_matrix,

    ConfusionMatrixDisplay,

    precision_score,

    recall_score,

    f1_score

)


data = pd.read_csv("emails.csv")


data = data.drop('Email No.', axis=1)


data, data.shape,data.head,data.tail,  data.info(),data.describe()


data['Prediction'].value_counts()


X = data.drop('Prediction', axis=1)

y = data['Prediction']
```

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20)

neigh = KNeighborsClassifier(n_neighbors=2)
neigh.fit(X_train, y_train)

y_pred = neigh.predict(X_test)

neigh.score(X_train, y_train)
neigh.score(X_test, y_test)

print("ConfusionMatrix:")
cm = confusion_matrix(y_test, y_pred)
cm

mat = ConfusionMatrixDisplay(confusion_matrix=cm)
mat.plot()
plt.show()

print(classification_report(y_test, y_pred))

print("accuracy_score:")
accuracy_score(y_test, y_pred)

print("precision_score:")
precision_score(y_test, y_pred)

print("recall_score:")
recall_score(y_test, y_pred)
```

```
print("Error:")
1 - accuracy_score(y_test, y_pred)

from sklearn.svm import SVC
SVM = SVC(gamma='auto')
SVM.fit(X_train, y_train)

SVM.score(X_train, y_train)
SVM.score(X_test, y_test)

SVM.score(X_train, y_train)
SVM.score(X_test, y_test)

print("ConfusionMatrix:")
cm = confusion_matrix(y_test, y_pred)
cm

mat = ConfusionMatrixDisplay(confusion_matrix=cm)
mat.plot()
plt.show()

print(classification_report(y_test, y_pred))
```

practical no :- 3

Problem Statement : Implement Gradient Descent Algorithm to find the locafunction.

```
# Import library
import matplotlib.pyplot as plt
```

```
 # Define the function and its derivative
def df(x):
    return (x + 3)**2
    def df(x):
        return 2 * (x + 3)

# Initialize parameters
x_old = 2          # Starting point
learning_rate = 0.1
precision = 0.00001
max_iterations = 1000

# Store for plotting
x_values = []
y_values = []
for i in range(max_iterations):
    gradient = df(x_old)
    x_new = x_old - learning_rate * gradient
    x_values.append(x_new)
    y_values.append(f(x_new))
    if abs(x_new - x_old) < precision:
        break
        x_old = x_new
        print(f"Local minima occurs at x = {x_new:.5f}")
        print(f"Minimum value of function y = {f(x_new):.5f}")

# Cell 2: gradient descent
x_values = []
y_values = []
```

```
for i in range(max_iterations):
    gradient = df(x_old)
    x_new = x_old - learning_rate * gradient
    x_values.append(x_new)
    y_values.append(f(x_new))
    if abs(x_new - x_old) < precision:
        break
    x_old = x_new

print(f"Local minima occurs at x = {x_new:.5f}")
print(f"Minimum value of function y = {f(x_new):.5f}")
```

practical no :- 4

Title:-ImplementK-NearestNeighborsalgorithmondiabetes.csvdataset.Compute confusion matrix, accuracy, error rate, precision and recall on the given dataset.

Dataset :- https://www.kaggle.com/datasets/abdallamahgoub/diabetes

```
import pandas as pd
import seaborn as sns

df = pd.read_csv("diabetes.csv")

df , df.head() , df.tail() , df.describe() , df.info() ,df.shape

x = df.drop('Outcome', axis=1)
y = df['Outcome']

sns.countplot(x=y);

y.value_counts()

from sklearn.preprocessing import MinMaxScaler
```

```
scaler = MinMaxScaler()
x_scaled = scaler.fit_transform(x)

from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split( x_scaled, y, random_state=0, test_size=0.25)

x.shape

x_train.shape

x_test.shape

from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(n_neighbors=5)

knn.fit(x_train, y_train)

from sklearn.metrics import accuracy_score, ConfusionMatrixDisplay
from sklearn.metrics import classification_report

y_pred = knn.predict(x_test)

ConfusionMatrixDisplay.from_predictions(y_test, y_pred)

print(classification_report(y_test, y_pred))

import matplotlib.pyplot as plt
import numpy as np
```

```
error = []
for k in range(1, 41):
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(x_train, y_train)
    pred = knn.predict(x_test)
    error.append(np.mean(pred != y_test))

error

plt.figure(figsize=(16,9))
plt.xlabel('value of k')
plt.ylabel('error')
plt.grid()
plt.xticks(range(1,41))
plt.plot(range(1,41), error, marker='.')

knn = KNeighborsClassifier(n_neighbors=33)

knn.fit(x_train, y_train)

y_pred = knn.predict(x_test)

print(classification_report(y_test, y_pred))
```

practical no :- 5

Title:-ImplementK-Meansclustering/hierarchicalclusteringon sales_data_sample.csv dataset. Determine the number of clusters using the elbow method.

Dataset :- https://www.kaggle.com/datasets/kyanyoga/sample-sales-data

```
import pandas as pd
import numpy as np
```

```
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from scipy.cluster.hierarchy import dendrogram,linkage,fcluster

df = pd.read_csv("sales_data_sample.csv", encoding="latin1")
print("Dataset loaded successfully")
print(df.head())

num_df=df.select_dtypes(include=[np.number])

num_df=num_df.dropna()

scaler=StandardScaler()
X_scaled=scaler.fit_transform(num_df)

inertia=[]
K=range(1,11)

for k in K:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_scaled)
    inertia.append(kmeans.inertia_)

plt.figure(figsize=(8,5))
plt.plot(K, inertia, 'bo-')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Inertia')
```

```
plt.title('Elbow Method For Optimal k')
plt.show()

optimal_k=3
kmeans = KMeans(n_clusters=optimal_k, random_state=42, n_init=10)
clusters = kmeans.fit_predict(X_scaled)

df['KMeans_Cluster'] = clusters
print("\nKMeans cluster counts:")
print(df['KMeans_Cluster'].value_counts())

pca=PCA(n_components=2)
X_pca=pca.fit_transform(X_scaled)

plt.figure(figsize=(8,6))
sns.scatterplot(x=X_pca[:,0], y=X_pca[:,1], hue=df['KMeans_Cluster'], palette="Set2", s=80, edgecolor="k")
plt.title("K-Means Clusters (2D PCA Projection)")
plt.show()

from sklearn.cluster import AgglomerativeClustering

# Use the same data you applied PCA on (e.g., X_scaled)
hc = AgglomerativeClustering(n_clusters=3, linkage='ward')
df['Hierarchical_Cluster'] = hc.fit_predict(X_scaled)

plt.figure(figsize=(8,6))
sns.scatterplot(x=X_pca[:,0], y=X_pca[:,1],
                hue=df['Hierarchical_Cluster'],
                palette="Set1", s=80, edgecolor="k")
plt.title("Hierarchical Clusters (2D PCA Projection)")
```

```
plt.show()
```